

18-Obfuscation - The Egg Shell File

Introduction

The Story



WareVille no se siente bien desde que apareció el agujero de gusano. Todos en TBFC están en alerta máxima: los sistemas se están volviendo locos, los paneles están aumentando y las alertas de SOC se han disparado sin parar. En medio del caos, McSkidy mantiene su atención en una alerta particular que llamó su atención: un correo electrónico que se hace pasar por northpole-hr. Está lleno de emojis de zanahorias, pero lo extraño es esto: no existe un departamento de recursos humanos en el Polo Norte. El HR de TBFC está en el Polo Sur.

Profundizando más, encontró que se descargó un pequeño archivo PowerShell de ese correo electrónico. Entre el código hay cadenas aleatorias de caracteres, todo un galimatías, nada legible de un vistazo.

McSkidy sabe que los actores maliciosos a menudo ocultan códigos y datos mediante una técnica llamada ofuscación. ¿Pero qué es realmente? ¿Y cómo podemos descifrarlo?

Objetivos de aprendizaje

- Aprenda sobre la ofuscación, por qué y dónde se utiliza.
- Conozca la diferencia entre codificación, cifrado y ofuscación.
- Aprenda sobre la ofuscación y las técnicas comunes.
- Utilice CyberChef para recuperar texto sin formato de forma segura.

Alternatively, you can use the credentials below to connect to the target machine via RDP from your own THM VPN connected machine:

Credentials

Only needed if you are using your own [THM VPN](#) connected machine.

Username	Password	IP address	Connection via
 Administrator	 @TryObfusc4t3M393!	 MACHINE_IP	 RDP

Answer the questions below

Start the target machine and continue with the next task.
No answer needed

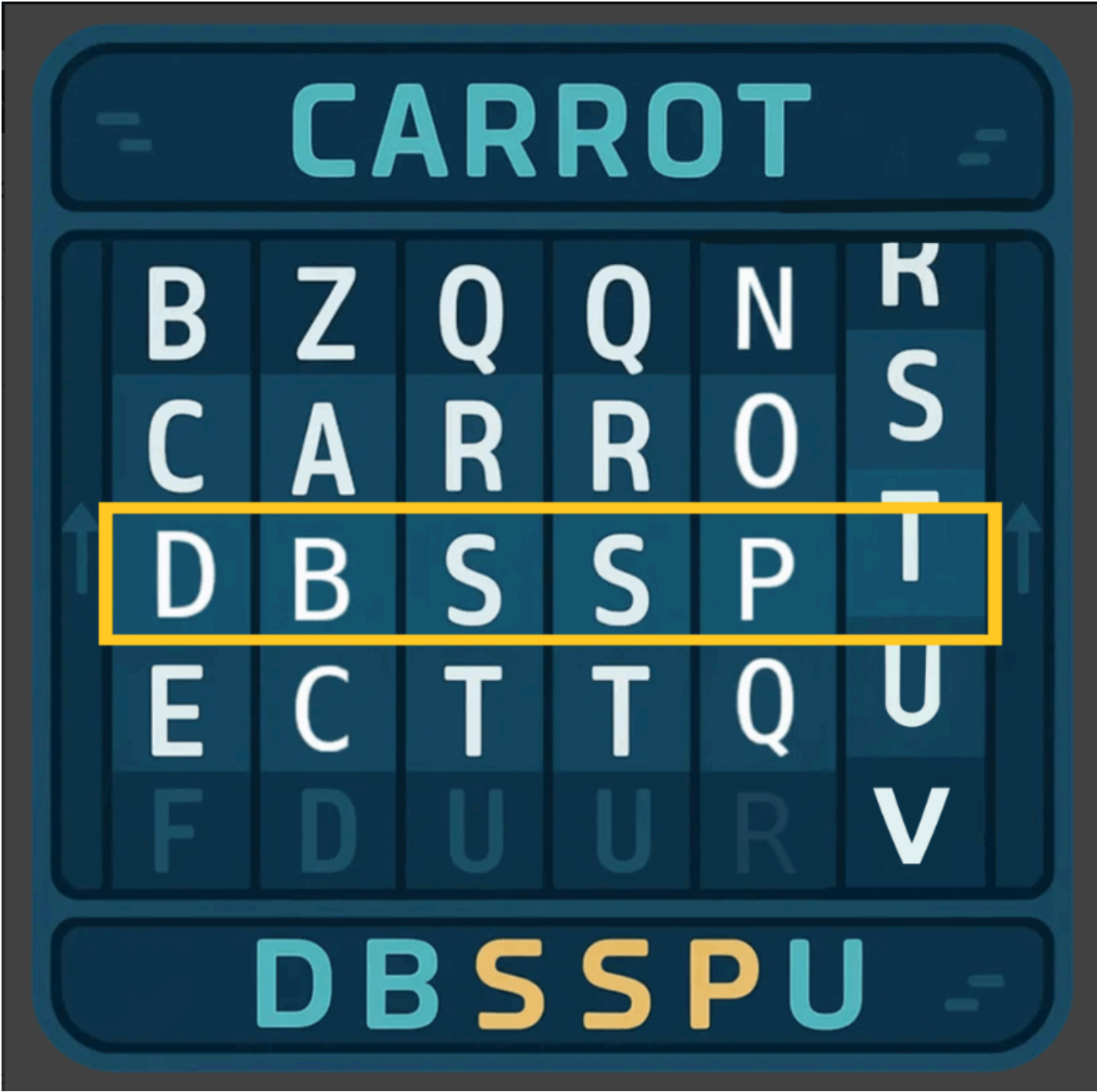
Obfuscation & Deobfuscation

Entendiendo el galimatías

La ofuscación es la práctica de dificultar la lectura y el análisis de los datos. Los atacantes lo utilizan para evadir la detección básica y retrasar las investigaciones.

Digamos que una herramienta de seguridad bloqueará automáticamente cualquier archivo que contenga la palabra "the bandit yeti". Para evadir esto, los atacantes pueden utilizar una técnica de cifrado simple llamada "ROT1" para ocultar la cadena.

ROT1 es una técnica de cifrado que avanza cada letra 1 en el alfabeto. La letra "a" se convierte en "b", "b" se convierte en "c", y así sucesivamente. Usando este cifrado, "carrot coins go brr" se convierten en "dbsspu dpjot hp css".



Esto puede complicarse aún más si se utiliza ROT13, que hace avanzar a los personajes 13 espacios.

Ofuscación en el mundo real

Hasta ahora, lo hemos mantenido simple. En el mundo real, encontrarás una gama más amplia de técnicas.

Una técnica un poco más complicada sería utilizar XOR. Cada carácter se representa como un byte y cada byte se combina con una clave mediante la operación matemática XOR. No es necesario entrar en detalles sobre cómo funciona. Lo único que debes saber es que esta operación cambia los bytes en bytes diferentes, lo que en ocasiones hace que el texto resultante acabe con símbolos poco comunes o caracteres no legibles.

Ofuscar con XOR no es algo que se pueda hacer fácilmente de forma manual. Pero, afortunadamente, existen herramientas disponibles que pueden ayudarnos con esto. Una de esas herramientas ampliamente utilizadas se llama CyberChef.

Intentemos ofuscar una cadena usando XOR siguiendo los pasos a continuación:

1. Abra CyberChef visitando este [enlace](#) en su navegador.
2. En la parte superior derecha, verá un cuadro de Entrada. Aquí es donde pegas el texto que deseas cambiar/ofuscar. En nuestro caso, utilice "carrot supremacy".
3. En el panel izquierdo, debajo de Operaciones, busque "XOR" y arrástrelo hacia el centro, debajo de Receta. Esto le indica a CyberChef que aplique esa técnica a su texto de entrada.
4. Cada operación tiene configuraciones que puede ajustar. En nuestro caso, configuramos la clave en "a". Asegúrese de que el menú desplegable junto a Clave esté configurado en HEX.
5. El resultado ofuscado debería aparecer automáticamente en la sección Salida. Si no es así, haga clic en ¡HORNEAR! en la parte inferior de la pantalla.

Como puedes ver arriba, carrot supremacy en hilo ahora se convierte en `ikxxe~*y zxogkis`.

Detección de patrones

Lo bueno de las técnicas de ofuscación conocidas es que es fácil revertirlas una vez que descubres la técnica utilizada.

Puede utilizar estas pistas visuales rápidas para adivinar la técnica de ofuscación utilizada:

- ROT1: las palabras comunes parecen "a una letra de distancia", los espacios permanecen iguales. Bastante fácil de detectar.
- ROT13 - Busca palabras de tres letras. Los más comunes como "the" convertido en "gur". Y "and" se convierte en "naq". Los espacios siguen siendo los mismos.
- Base64: cadenas largas que contienen principalmente caracteres alfanuméricos (es decir, A-Z, a-z, 0–9), a veces con + o /, y a menudo terminan en = o ==.
- XOR: un poco más complicado. Parecen símbolos aleatorios pero mantienen la misma longitud que el original. Si se reutilizó un breve secreto, es posible que observe una pequeña repetición cada pocos caracteres.

Una vez que sepa qué cifrado se utilizó, puede volver a CyberChef, pero esta vez utilice una operación que produce el efecto inverso. Por ejemplo, para desofuscar Base64 se utiliza "From Base64" en lugar de To Base64.

Recipe

From Base64

Alphabet
A-Za-z0-9+/=

☒ Remove non-alphabet chars

☐ Strict mode

XOR

Key
X

HEX

Scheme
Standard

☐ Null preserving

Gunzip

Input

H4sIADKZ42gA/32PT2sqQRDE7/MpitGDgrPEJJcXy0Gha1xwVaLwyLvI6rbuhP3HTHswm/3uzmggIQfrUD3VzI/u7iDXljepNkFth6KDmYswNBf2R20oZ0yrPCUDXSKB1UWdEzjZ5hQI8c9oJrU4cn1kyPXbMgRW0X/nF8WLcTSJwvFX9Jr/jUP5i1N0gPeLfjx

sec 404 1 83

Raw Bytes

LF

Output

```
# list_dir.ps1
# Lists the current folder in a simple table.

Write-Output "TYPE    SIZE      MODIFIED      NAME"
Write-Output "-----"
-"

Get-ChildItem | ForEach-Object {
    $type = if ($_.PSIsContainer) { "DIR " } else { "FILE" }
    $size = if ($_.PSIsContainer) { "--" } else { $_.Length }
    $date = $_.LastWriteTime.ToString("yyyy-MM-dd HH:mm")
    "{0,-4} {1,9} {2} {3}" -f $type, $size, $date, $_.Name
}
```

Desenvolver el huevo de Pascua

Afortunadamente, McSkidy detectó algo fraudulento en el correo electrónico que recibimos. Extrajo un script de PowerShell del PDF y lo colocó en una máquina virtual aislada para descubrir qué está haciendo. Hay cierta confusión en este guión, ¿puedes ayudarla a entender de qué se trata?

Abra SantaStealer.ps1, ubicado en el escritorio de la máquina virtual en Visual Studio, haciendo doble clic en él (tarda un momento en abrirse) y navegue hasta la sección "Comenzar aquí" del script. Siga las instrucciones en los comentarios del código y guárdelo, luego ejecute el script desde una terminal PowerShell para obtener su primer indicador. Para abrir PowerShell, haga clic en Inicio (o presione la tecla de Windows), escriba "PowerShell" y presione Entrar. En la terminal de PowerShell, ejecute los dos comandos que se muestran en los comentarios del script.

```
PS C:\Users\Administrator> cd .\Desktop\
PS C:\Users\Administrator\Desktop> .\SantaStealer.ps1
```

Hay un segundo desafío esperándote. Esta vez, debes ofuscar la clave API del actor malicioso usando XOR. Siga las instrucciones de la Parte 2 en los comentarios del código y luego ejecute el script nuevamente para obtener su última bandera.

Answer the questions below

What is the first flag you get after deobfuscating the C2 URL and running the script?

Respuesta: THM{C2_De0bfuscation_29838}

File Edit Selection ... Search [Administrator]

Restricted Mode is intended for safe code browsing. Trust this window to enable all features. Manage Learn More

SantaStealer.ps1

C: > Users > Administrator > Desktop > SantaStealer.ps1

```
1  # Only edit where the TODOs say. Do not remove the validator call at the end.
2
3  # =====
4  # IGNORE THIS
5  # =====
6  $ErrorActionPreference = 'SilentlyContinue'
7
8  # =====
9  # Start here
10 # Part 1: Deobfuscation
11 # =====
12 # TODO (Step 1): Deobfuscate the string present in the $C2B64 variable and place the URL in the
13 # then run this script to get the flag.
14
15 $C2B64 = "aHR0cHM6Ly9jMi5ub3J0aHBvbGUudGhtL2V4Zmls"
16 $C2    = "<C2_URL_HERE>"
17 # =====
18 # Part 2: Obfuscation
19 # =====
20 # TODO (Step 2): Obfuscate the API key using XOR single-byte key 0x37 and convert to hexadecimal.
21 # then add the hex string to the $ObfAPIeEy variable.
22 # Then run this script again to receive Flag #2 from the validator.
23 $ApiKey = "CANDY-CANE-API-KEY"
24 $ObfAPIKEY = Invoke-XorDecode -Hex "<HEX_HERE>" -Key 0x37
25 # =====
26 # =====
27 function Decode-B64 {
28     param([Parameter(Mandatory=$true)][string]$S)
29     try { [System.Text.Encoding]::UTF8.GetString([Convert]::FromBase64String($S)) } catch { ""
30 }
31
32 function Invoke-XorDecode {
33     [CmdletBinding()]
34     param(
```


Recipe

From Base64

Alphabet

A-Za-z0-9+/=

☒ Remove non-alphabet chars

☒ Strict mode

Input

aHR0cHM6Ly9jMi5ub3J0aHBvbGUudGhtL2V4Zm1s

Output

https://c2.northpole.thm/exfil

File Edit Selection ... Search [Administrator]

Restricted Mode is intended for safe code browsing. Trust this window to enable all features. Manage Learn More

SantaStealer.ps1

C: > Users > Administrator > Desktop > SantaStealer.ps1

```
1 # Only edit where the TODOs say. Do not remove the validator call at the end.
2
3 # =====
4 # IGNORE THIS
5 # =====
6 $ErrorActionPreference = 'SilentlyContinue'
7
8 # =====
9 # Start here
10 # Part 1: Deobfuscation
11 # =====
12 # TODO (Step 1): Deobfuscate the string present in the $C2B64 variable and place the URL in the
13 # then run this script to get the flag.
14
15 $C2B64 = "aHR0cHM6Ly9jMi5ub3J0aHBvbGUudGhtL2V4Zm1s"
16 $C2 = "https://c2.northpole.thm/exfil"
17 # =====
```

PS C:\Users\Administrator\Desktop> .\SantaStealer.ps1
[i] incorrect XOR-obfuscated API hex.
[i] Operator session started
[*] Recon: collecting host and user context
[*] Stealing Santas presents list
[*] Preparing payload
[*] Contacting C2 endpoint
[i] Exfiltration attempted (no response)
[*] Establishing foothold
[*] Downloading payload...
THM{C2_Deobfuscation_29838}
PS C:\Users\Administrator\Desktop>

What is the second flag you get after obfuscating the API key and running the script again?

Respuesta: THM{API_Obfusc4tion_ftw_0283}

```
17 # =====
18 # Part 2: Obfuscation
19 # =====
20 # TODO (Step 2): Obfuscate the API key using XOR single-byte key 0x37 and convert to hexadecimal
21 # then add the hex string to the $ObfAPIeEy variable.
22 # Then run this script again to receive Flag #2 from the validator.
23 $ApiKey = "CANDY-CANE-API-KEY"
24 $ObfAPIKEY = Invoke-XorDecode -Hex "<HEX_HERE>" -Key 0x37
25 # =====
```

Recipe

XOR

Key
0x37

HEX

Scheme
Standard

☐ Null preserving

Input

<1: aHR0cHM6Ly9jMi5ub3J0aHBvbGUudGhtL2V4Zmls

×

2: CANDY-CANE-API-KEY

CANDY-CANE-API-KEY|

REC 18

1

Output

<Tab 1

2: tvysn tvyr vg~ |rn

|tvysnsubtvyrsubvg~sub |rn

Recipe

To Hex

Delimiter
Space

Bytes per line
0

Input

+

1: aHR0cHM6Ly9jMi5ub3J0aHBvbGUu...

×

2: CANDY-CANE-API-KEY

×

3: tvysn tvyr vg~ |rn

×

tvysnsubtvyrsubvg~sub |rn|

REC 18

1

Output

Tr Raw Bytes

<Tab 1

2: tvysn tvyr vg~ |rn

3: 74 76 79 73 6e 1a 74 76 79 72 1a 76 67 7e 1a 7c 72 6e...

74 76 79 73 6e 1a 74 76 79 72 1a 76 67 7e 1a 7c 72 6e

```
18 # Part 2: Obfuscation
19 # =====
20 # TODO (Step 2): Obfuscate the API key using XOR single-byte key 0x37 and convert to hexadecimal,
21 # then add the hex string to the $ObfAPIeEy variable.
22 # Then run this script again to receive Flag #2 from the validator.
23 $ApiKey = "CANDY-CANE-API-KEY"
24 $ObfAPIKEY = Invoke-XorDecode -Hex "74 76 79 73 6e 1a 74 76 79 72 1a 76 67 7e 1a 7c 72 6e" -Key 0x37
25 #
```

```
PS C:\Users\Administrator\Desktop> .\SantaStealer.ps1
[i] incorrect XOR-obfuscated API hex.
[i] Operator session started
[*] Recon: collecting host and user context
[*] Stealing Santas presents list .....
[*] Preparing payload
[*] Contacting C2 endpoint
[i] Exfiltration attempted (no response)
[*] Establishing foothold
[*] Downloading payload...
THM{C2_Deobfuscation_29838}
THM{API_Obfusc4tion_ftw_0283}
PS C:\Users\Administrator\Desktop>
```

If you want to learn more about Obfuscation, check out our [Obfuscation Principles](#) room!

no necesita respuesta